

AIDA vs GitHub Spec Kit

AIDA positioning · best-effort comparison — live source:
github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

The TL;DR: **Spec Kit and AIDA agree on the premise — specs live in git, as files, versioned alongside the code, and agents work from them.** They diverge on what happens *after* a feature is specced. Spec Kit's model is **spec → plan → tasks → implement, then the spec is a frozen artifact of that feature.** AIDA's model is **a living, cross-cutting graph of specs with stable IDs, typed relationships, and code↔spec traces that an orchestrator maintains over the project's whole life.** If you want a great first-feature scaffold, Spec Kit is excellent and far more widely adopted. If you want the specs to stay queryable, related, and enforced as the project grows, that's AIDA's bet.

This is the most important page in this directory: Spec Kit is AIDA's **nearest competitor**, and the one with by far the larger distribution.

What Spec Kit is

GitHub Spec Kit (github.com/spec-kit, ~113k stars per the 2026-07-07 roster, GitHub-first-party, MIT) is the leading open-source **spec-driven-development (SDD)** toolkit. The workflow, driven by slash commands inside your coding agent:

- `/speckit.constitution` — project-wide principles the agent must honor.
- `/speckit.specify` — turn a feature description into a structured spec (a `spec.md` under a numbered feature directory like `specs/001-feature/`).
- `/speckit.plan` — derive a technical plan from the spec.
- `/speckit.tasks` — break the plan into ordered tasks (T- ids).
- `/speckit.analyze` — a one-shot consistency check across `spec/plan/tasks`.
- `/speckit.implement` — execute the tasks.

It is genuinely good: zero-infrastructure, git-native, agent-agnostic (works in Claude Code, Copilot, Cursor, Gemini CLI...), and backed by GitHub. For a team that wants to go from idea to a well-structured first implementation, it is a strong, well-supported default.

Be precise about what it already has — overclaiming against Spec Kit is the easiest way to lose credibility:

- It **does** have IDs: FR-### (functional requirements), SC-### (success criteria), T-### (tasks) — *within a feature*.
- It **does** have a consistency check (/speckit.analyze) — *run on demand, across one feature's spec/plan/tasks*.
- It **does** version specs in git alongside the code — the same “filesystem is the API” rationale AIDA uses.

Where Spec Kit holds up (and you should just use it)

- **First-feature scaffolding.** Going from a paragraph to a structured spec → plan → tasks is exactly what it’s built for, and it does it well.
- **Single-feature scope.** When the unit of work is one feature at a time and features don’t densely cross-reference each other.
- **Distribution and trust matter to you.** It’s GitHub-first-party with a huge community. That is a real, rational reason to choose it, and AIDA should not pretend otherwise.
- **You want zero infrastructure.** No cache, no orphan branch, no MCP server — just markdown the agent reads.

If that’s your shape, **use Spec Kit**. AIDA’s added machinery wouldn’t earn its keep.

Where Spec Kit starts to crack — and what AIDA adds

The cracks all share one root: **Spec Kit’s specs are per-feature artifacts, not a project-wide graph**. Each lives in its own numbered directory; IDs are scoped to that feature; relationships across features live in prose or in someone’s head; the consistency check is a one-shot, not a maintained invariant.

Symptom as the project grows	Spec Kit	AIDA
<i>“What’s blocked across this whole epic?”</i>	No cross-feature relationship model to walk — each feature dir is an island	Typed BlockedBy/parent/child graph; aida graph / queue views walk it across the tree
<i>“Rename / merge this requirement”</i>	IDs are per-feature and positional (001-feature/); cross-references are string matches	Stable IDs resolve to the same UUID forever; references in code, commits, other specs don’t rot

Symptom as the project grows	Spec Kit	AIDA
<i>"Does this code still trace to its spec?"</i>	No code↔spec trace enforcement; drift between an implemented feature and its spec.md is undetected after /implement	// trace:SPEC-ID comments, commit-trailer linkage, and a lifecycle that bumps state on merge
<i>"Show me everything tagged auth, across all features"</i>	grep over markdown returns text, not records	Cache-backed query (aida list/search --tags) returns records in sub-ms
<i>"What's the status of everything, and what changed?"</i>	Status isn't a maintained property; no per-spec history	Lifecycle state machine + auto-bump on merge + per-spec history audit array
<i>"Let an orchestrator drain a queue of specs across agents"</i>	Not its model — /implement runs one feature for one agent	Orchestrated multi-phase drain with spec-grounded escalation + shelving across a vendor-neutral fleet
<i>"Query the spec graph from any agent, any vendor"</i>	Markdown the agent reads in-prompt	Token-efficient CLI (the primary agent surface) reads the graph on any vendor; an MCP server exposes it as typed tools/resources for MCP clients

The honest framing: **Spec Kit standardizes how an agent *produces* a feature's specs. AIDA is the graph *underneath* that keeps every feature's specs stable, related, traced, and queryable for the life of the project — and writes the agent-facing context files (AGENTS.md and friends) from that graph.**

The honest caveats (don't let AIDA overclaim)

- **Distribution is Spec Kit's, overwhelmingly.** ~100× the adoption and GitHub's backing. AIDA's edge is the structured-graph layer, not reach — and reach matters. AIDA's own risk is distribution, not differentiation.
- **Spec Kit is not standing still.** Its roadmap currently aims at GitHub Issues integration and richer agent prompts — *not* (as of this snapshot) stable cross-cutting IDs, a relationship graph, or trace enforcement. If that changes, this page changes. (Tripwire tracked in docs/competitive-analysis/.)
- **AIDA costs more to adopt.** Orphan branch, cache rebuilds, two-leg sync, an MCP server — a real conceptual + ops tax Spec Kit doesn't levy. AIDA earns it back only once the project is big and cross-linked enough that the graph pays for itself.
- **They're composable, not mutually exclusive.** You can scaffold a feature with Spec Kit and let AIDA hold the cross-feature graph + traces + lifecycle. AIDA is

the system-of-record layer, not a replacement for the scaffolder. The concrete composition seam is `aida plan scan <SPEC>` — a read-only context-grounding pass that summarizes the current API surface (from the trace graph), the architectural constraints, and likely-stale assumptions before you generate or import an artifact. Run it first, hand its summary to Spec Kit / OpenSpec as grounding context, then --attach the result so the imported spec records what the tree actually looked like at plan time.

When to use which

Use Spec Kit when...	Use AIDA when...
You want a great first-feature spec→plan→tasks scaffold	You want specs to stay a living, cross-cutting graph
Work is one feature at a time, loosely cross-referenced	Features densely cross-reference; you need a relationship graph
Zero infrastructure is a hard requirement	You want stable IDs + trace enforcement + a query cache + MCP
GitHub-first-party + huge community is decisive	Multi-vendor portability + an orchestrated drain is decisive
The spec is done when the feature ships	The spec graph must outlive every individual feature

Bottom line

Spec Kit proved the premise AIDA also bets on — **specs belong in git, as files, agent-readable** — and it owns the distribution for first-feature scaffolding. AIDA's claim is narrower and deeper: **once a project has enough specs that the relationships between them, their stable identity, and their traceability to code start to matter more than producing any single one, you want a maintained graph, not a folder of frozen per-feature artifacts.** That graph — stable IDs, typed relationships, enforced traces, a lifecycle, an MCP surface, portable across every vendor because it lives in git — is what AIDA adds on top of the floor Spec Kit established.

See also: [vs-karpathy-md.md](#) (the structured-markdown floor below both tools) and [docs/competitive-analysis/2026-05-31-round2-moat-gaps-moves.md](#) (the full competitive picture).